

Министерство науки и высшего образования
Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

Вариант №10
по лабораторной работе 5
по курсу «Программирование»
на тему «Процедуры»

Выполнили:

студенты группы 23ВВВ4

Калинин А.А.

Беглов И.В.

Принял:

Слепцов Н. В.

Голотенков Н.О.

Пенза 2024

Название

Процедуры

Цель работы

Получение навыков разработки структурной организации ассемблерных программ и ее реализации на основе аппарата процедур, изучение способов организации связи по данным между процедурами.

Лабораторное задание

Разработать программу вычисления функции $y=f(a,b,c,d,e)$. Значения аргументов a, b, c, d, e должны вводиться с клавиатуры ПЭВМ. Значение функции y следует вывести на экран дисплея.

Для ввода-вывода числовых данных использовать следующий формат: ЗнЦЦЦ...Ц, где Ц – цифра (0...9), Зн – знак (+,-). Предусмотреть в программе вывод текста, содержащего вычисляемую функцию и формат аргументов, а также выдачу запроса на ввод каждого аргумента. Вычисление функции и преобразование вводимых и выводимых данных оформить в виде процедур: func – вычисление заданной функции, ascbin – преобразование числа из строки ASCII-кодов в двоичное число, binasc – преобразование числа, представленного в виде двоичного кода, в строку ASCII кодов.

10)	21	4/3	8/7	6/5	w
-----	----	-----	-----	-----	---

Листинг

```
#include <stdio.h>
#include <conio.h>
#include <locale>

#define _CRT_SECURE_NO_WARNINGS

char* paramStr;

//Вычисление функции
__declspec(naked)
void func() {
    __asm {
        xor eax, eax
        xor ebx, ebx
        xor ecx, ecx

        pop edi
        pop eax
        pop eax
        pop ebx
    }
}
```

```

        imul eax, ebx

        xor ebx, ebx
        pop ebx
        cdq
        idiv ebx

        // Загрузка d в ebx
        xor ebx, ebx
        pop ebx

        // Загрузка e в ecx

        pop ecx

        imul ebx, ecx

        sub eax, ebx

        // Сохранение результата в адресе, указанном в edi
        movzx edi, eax
        ret 24
    }
}
// Вызов - данные, возврат - стек
__declspec(naked)
void ascbin() {
    _asm {

        mov edi, [esp+4]

        xor ecx, ecx
        xor edx, edx

        gotoLastChar :

        cmp[edi], '0'
        je processStr

        inc edi

        jmp gotoLastChar

        processStr :

        dec edi

        strToInt :

        xor ebx, ebx

        mov bl, [edi]

        cmp bl, '+'
        je returnValue

        cmp bl, '-'
        je signCorrection

        sub bl, '0'

        mov al, 1

        cmp cl, 0
        je powBypass

        push edx

```

```

        push ecx

        powLoop :

mov dl, 10

        imul dl

        loop powLoop

        pop ecx
        pop edx

        powBypass :

imul bl

        add dx, ax

        inc cx

        dec edi

        jmp strToInt

        signCorrection :

neg edx

        returnValue :

mov[esp + 8], edx

        ret 4
    }
}

// Вызов - регистр, возврат - регистр
__declspec(naked)
void binasc() {
    // al - number
    // edi - array
    __asm {

        xor ecx, ecx

        mov cl, 0

        cmp al, 0
        jge intToString

        mov[edi], '-'

        inc edi

        neg al

        intToString :

        cmp al, 0
        je loadFromStack

        mov dl, 10
        div dl

        xor edx, edx

        mov dl, ah
    }
}

```

```

        xor ah, ah

        push edx

        inc cl

        jmp intToString

    loadFromStack :

xor edx, edx
pop edx

    add dl, '0'

    mov[edi], dl

    inc edi

    loop loadFromStack

    mov[edi], '\0'

    ret
}

}

void main() {

    char aStr[10], bStr[10], cStr[10], dStr[10], eStr[10];
    char a, b, c, d, e, y;
    char asciiArray[10];

    strcpy_s(aStr, "+2");
    strcpy_s(bStr, "+2");
    strcpy_s(cStr, "+4");
    strcpy_s(dStr, "-10");
    strcpy_s(eStr, "+5");

    __asm {

        //Перевод числа a
        lea edi, aStr
        push 0
        push edi
        call ascbin
        pop ebx
        mov a, bl

        //Перевод числа b
        lea edi, bStr
        push 0
        push edi
        call ascbin
        pop ebx
        mov b, bl

        //Перевод числа c
        lea edi, cStr
        push 0
        push edi
        call ascbin
        pop ebx
        mov c, bl

        //Перевод числа d
        lea edi, dStr

```

```

push 0
push edi
call ascbin
pop ebx
mov d, bl

//Перевод числа e
lea edi, eStr
push 0
push edi
call ascbin
pop ebx
mov e, bl

//Передача в функцию переменной e
xor ebx, ebx
mov bl, e
push ebx

//Передача в функцию переменной d
xor ebx, ebx
mov bl, d
push ebx

//Передача в функцию переменной c
xor ebx, ebx
mov bl, c
push ebx

//Передача в функцию переменной b
xor ebx, ebx
mov bl, b
push ebx

//Передача в функцию переменной a
xor ebx, ebx
mov bl, a
push ebx

//Передача в функцию переменной y
lea edi, y
push edi

call func

xor eax, eax

mov al, y

lea edi, asciiArray

call binasc

}

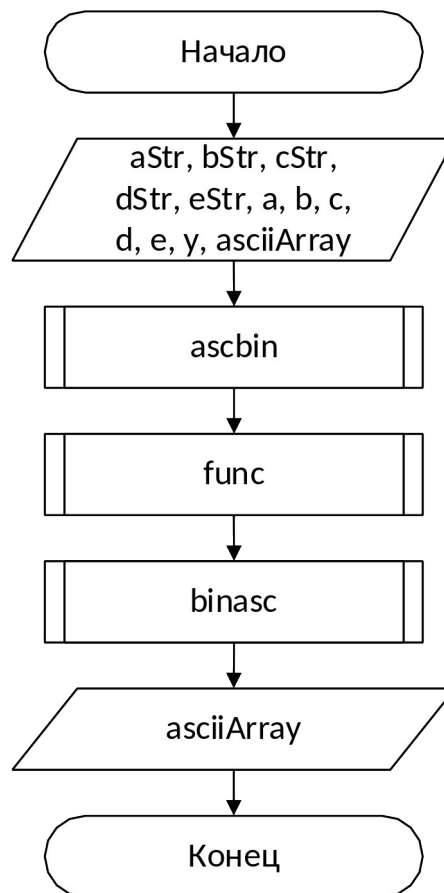
printf("y = a * b / c - d * e\n");
printf("\na = %d\nb = %d\nc = %d\nd = %d\ne = %d\n", a, b, c, d, e);
printf("\ny = %s", asciiArray);

_getch();
}

```

Пояснительный текст к программе
 Переменные a, b, c, d, e хранят числовые аргументы функции, переменные aStr, bStr, cStr, dStr, eStr хранят строковые аргументы функции, y - хранит численный результат, asciiArray - хранит строковый результат, paramStr нужна для промежуточного хранения строк.

Схема программы



Результат работы программы

Результаты работы программы показаны на рисунке 1.

```
C:\Users\Иван\source\repos\ x + v
y = a * b / c - d * e
a = 2
b = 2
c = 4
d = -10
e = 5
y = 51|
```

The screenshot shows a terminal window with a dark background. The first line is the program's formula: `y = a * b / c - d * e`. Below it, the values of the variables are assigned: `a = 2`, `b = 2`, `c = 4`, `d = -10`, and `e = 5`. The final line shows the calculated result: `y = 51|`, with a cursor at the end of the line.

Рисунок 1 – результат работы программы 1

Протокол трассировки программы

На рисунке 2 показан протокол трассировки программы.

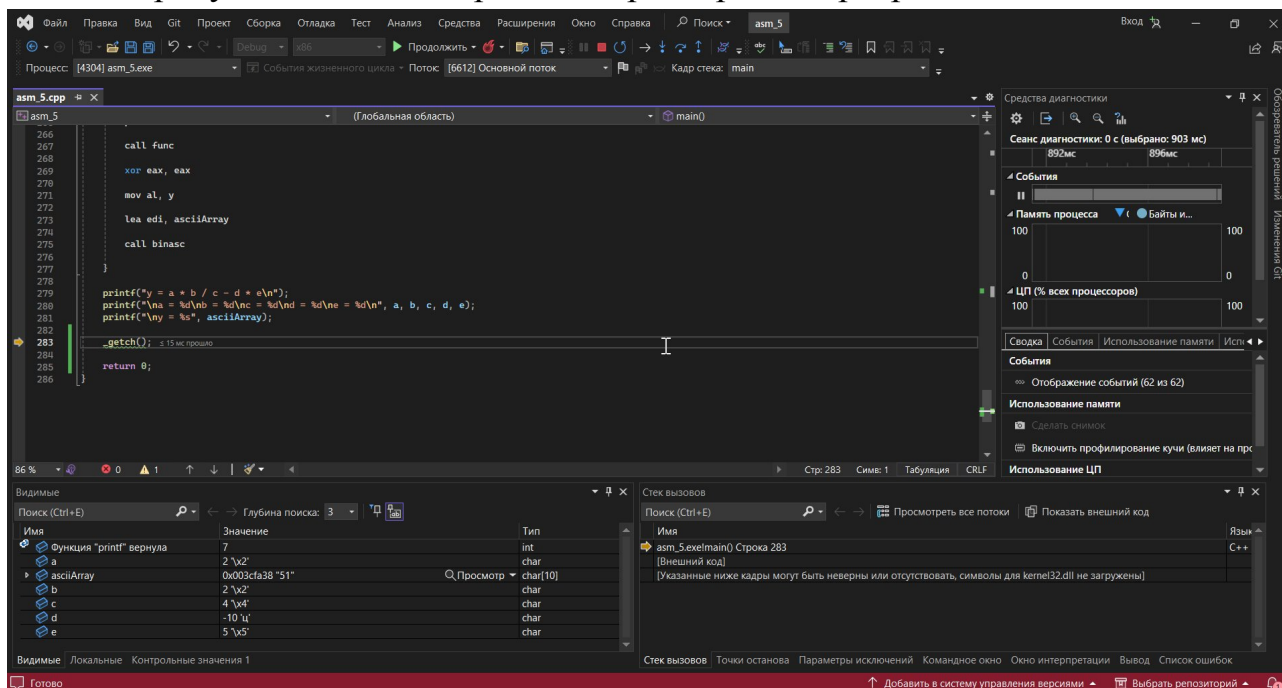


Рисунок 2 – Протокол трассировки программы 1

Вывод

В ходе выполнения лабораторной работы были получены навыки разработки структурной организации ассемблерных программ и ее реализации на основе аппарата процедур, изучены способы организации связи по данным между процедурами.